UCS503 - Software Engineering

FlashCart: Unified Mall Checkout

Scan products across every showroom, pay once, get one GST invoice

Submitted by:

1024031127 Pranay Nagpal
1024031145 Suryansh Jain
1024031144 Srijan Tanwar

Group: 3C82 BE Third Year, CSE

Submitted to:

Jhonsy Bansal

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
- 1.1 Background and Context . . . 3
  - 1.1.1 Problem Statement . . . 3
- 1.2 Project Objectives . . . 3

2 Methodology and Design **4**
- 2.1 System Architecture . . . 4
  - 2.1.1 High-Level Design . . . 4
  - 2.1.2 Technical Stack . . . 4
- 2.2 Implementation Details . . . 5
  - 2.2.1 Module 1: Customer App and Barcode Scanning . . . 5
  - 2.2.2 Module 2: GST Pricing and Unified Checkout . . . 5
  - 2.2.3 Module 3: Cash Desk and End-of-Day Drawer . . . 5
  - 2.2.4 Module 4: Authentication and Database . . . 6
- 2.3 Software Development Life Cycle . . . 6
- 2.4 UML Diagrams . . . 6
  - 2.4.1 Use Case Diagram . . . 6
  - 2.4.2 UML Sequence Diagram . . . 7
  - 2.4.3 Activity Diagram . . . 7
  - 2.4.4 Class Diagram . . . 7

3 Results and Evaluation **12**
- 3.1 Testing Methodology . . . 12
- 3.2 Performance Metrics . . . 12
- 3.3 Analysis . . . 12

4 Conclusion **13**
- 4.1 Summary of Work . . . 13
- 4.2 Key Findings . . . 13
- 4.3 Future Work and Recommendations . . . 13
- 4.4 Final Remarks . . . 13

Chapter 1

Introduction

1.1 Background and Context

A modern Indian mall houses dozens of independent showrooms (apparel, electronics, food, jewellery, beauty), and each runs its own billing counter. A shopper who buys a shirt, earbuds and a coffee queues three times, pays three times and leaves with three bills, each with its own GST calculation. **FlashCart** replaces these counters with one digital checkout. The shopper signs in with a mobile number and scans the barcode on any product tag with the phone camera. Every item from every showroom lands in one *unified bag*, which is paid once (UPI or card through Razorpay, or cash at a central desk) for a single GST invoice.

1.1.1 Problem Statement

Multi-showroom retail suffers from repeated queues and payments per store, fragmented receipts that are hard to audit for GST, manual cash handling with change errors and no reliable end-of-day drawer total, and carts that are lost when a device changes. The project builds a web-based unified checkout that removes these problems without new hardware in the showrooms, using the EAN-13 barcodes already printed on retail tags.

1.2 Project Objectives

1. **Objective 1:** Phone-number sign-in and camera scanning of EAN-13 / UPC / Code 128 barcodes, adding an item to the bag in under one second.
2. **Objective 2:** An exact multi-slab GST bill (5%, 12%, 18%) across all showrooms, including tags whose MRP already includes GST.
3. **Objective 3:** One payment: Razorpay (UPI / card) with server-side HMAC-SHA256 verification, or cash at a desk.
4. **Objective 4:** An email-and-password cash desk that shows the change to return and the cash left in the drawer at day end.
5. **Objective 5:** Persist all data in a database and cover the business rules with automated tests, delivered in three iterations (Section 2.3).

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

FlashCart follows a three-tier client–server architecture (Figure 2.1). Two single-page applications run in the browser: the *customer app* on the shopper's phone, and the *cash desk* on a counter tablet or laptop. Both talk JSON over HTTPS to one Express REST API, and every protected call carries a signed bearer token. The API owns all business rules: pricing, stock, payment verification and cash accounting. Clients never send amounts, they only send intents. Durable data lives in an SQLite database, while short-lived carts and gateway orders are kept in memory. Online payments go through Razorpay [?].

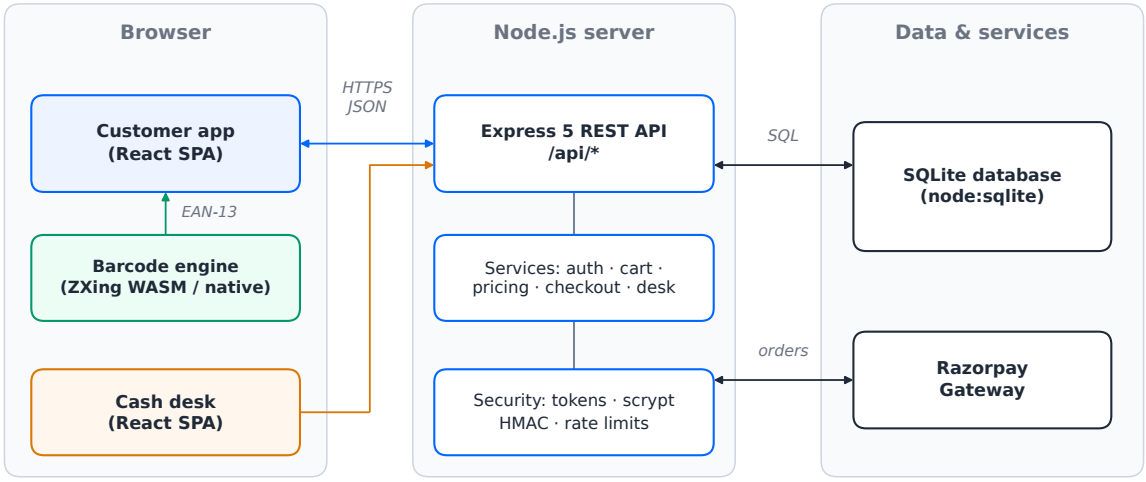


Figure 2.1: High-level three-tier architecture of FlashCart

2.1.2 Technical Stack

- **Framework/Language:** JavaScript (ES2022). React 19 with Vite 8 for the two web apps [?, ?]; Node.js 22+ with Express 5 for the REST API [?].
- **Database:** SQLite through Node's built-in `node:sqlite` module [?]: one file, no separate database server.

- **Tools:** Git and GitHub, npm, Vite HTTPS dev server for testing on phones, Razorpay test-mode dashboard, oxlint, and the built-in `node:test` runner.
- **Libraries:** Tailwind CSS 4, Lucide icons and Axios for the UI; ZXing-C++ compiled to WebAssembly for barcode reading [**?**, **?**]; JsBarcode for bill barcodes; the Razorpay Node SDK, uuid and dotenv on the server.

2.2 Implementation Details

2.2.1 Module 1: Customer App and Barcode Scanning

After a phone-number sign-in the customer lands on a home screen (bag total, showrooms, recent receipts). The camera opens only on the *Scan* screen. It uses the rear camera at 1920×1080 and decodes a wide central band every 90 ms, plus the full frame every fourth pass for tags held off-centre. Decoding uses the native `BarcodeDetector` where available (Chrome) [**?**], and otherwise ZXing-C++ compiled to WebAssembly [**?**], which the app serves itself and loads only when the scanner opens. Only linear formats are enabled (EAN-13/8 [**?**], UPC, Code 128/39, ITF), so a QR code on the same tag is never picked up. Repeat reads within 2.2 s are ignored. A beep, a haptic tick and an *Undo* action confirm each scan, and manual entry and catalog search work without a camera.

2.2.2 Module 2: GST Pricing and Unified Checkout

All arithmetic uses integer paise. For GST-exclusive prices, tax is round(subtotal × rate). Real tags print an MRP that *includes* tax, so the line total is exactly MRP × qty and the taxable value is MRP ÷ (1 + rate). For example, a Rs. 2999 polo bills as Rs. 2541.53 taxable + Rs. 457.47 GST. Tax is summed per slab, split into CGST and SGST, and rounded to the rupee, and invoice numbers run per financial year (`FC/2627/000001`). For online payment the server prices the bag, creates the Razorpay order, and verifies HMAC-SHA256(*order_id* | *payment_id*) in constant time [**?**, **?**] before saving the invoice and reducing stock in one SQLite transaction. Replayed callbacks return the same receipt.

2.2.3 Module 3: Cash Desk and End-of-Day Drawer

Choosing cash issues a *pending* invoice with a Code 128 bill barcode. The cashier (signed in by email and password) records the opening cash, then finds the bill by scanning it or typing the customer's phone number. Entering the cash received shows the *change to return* with a note split: Rs. 5000 for a Rs. 3894 bill gives Rs. 1106 as $2 \times 500 + 100 + 5 + 1$. Short payments are rejected. The drawer panel shows *opening cash + cash sales*. At closing, the counted cash is compared with that figure

to report any overage or shortage, and a printable day report is produced. The customer's receipt turns *Paid* as soon as the cash is recorded.

2.2.4 Module 4: Authentication and Database

Customers sign in with a 10-digit mobile number; cashiers with an email and a salted scrypt password hash [**?**]. Sessions are HMAC-signed tokens (30 days for customers, 12 hours for cashiers), role guards keep each role in its own API area, and login endpoints are rate-limited. Table 2.1 lists the schema.

Table	**Key columns**
customers	phone (unique), created_at, last_login_at
cashiers	email (unique), name, password_hash, active
products	barcode (EAN-13), name, cost, mrp, gst, type, hsn, showroom, stock
transactions	invoice no., customer, grand total, method, status, cash received, change
cash_shifts	cashier, opening cash, opened/closed at, counted cash

Table 2.1: SQLite database schema

2.3 Software Development Life Cycle

We followed an **iterative and incremental (agile)** SDLC [**?**] (Figure 2.2). Each iteration went through requirements, design, build, test, demo and review, and the review feedback set the next scope: iteration 1 (weeks 1–6) built the MVP, iteration 2 (weeks 7–8) added sign-in, SQLite and the cash desk, and iteration 3 (weeks 9–10) added the barcode engine, real products and MRP pricing.

2.4 UML Diagrams

The design is modelled with four UML [**?**] views: use case, sequence, activity and class diagrams.

2.4.1 Use Case Diagram

Figure 2.3 shows two primary actors and one external system. The *Customer* signs in, scans or browses (both include *Add item to bag*), manages the bag and checks out. *Pay online* and *Pay cash at the desk* extend Checkout, and online payment always includes signature verification with *Razorpay*. The *Cashier* signs in, starts the day, finds bills, collects cash and closes the day.

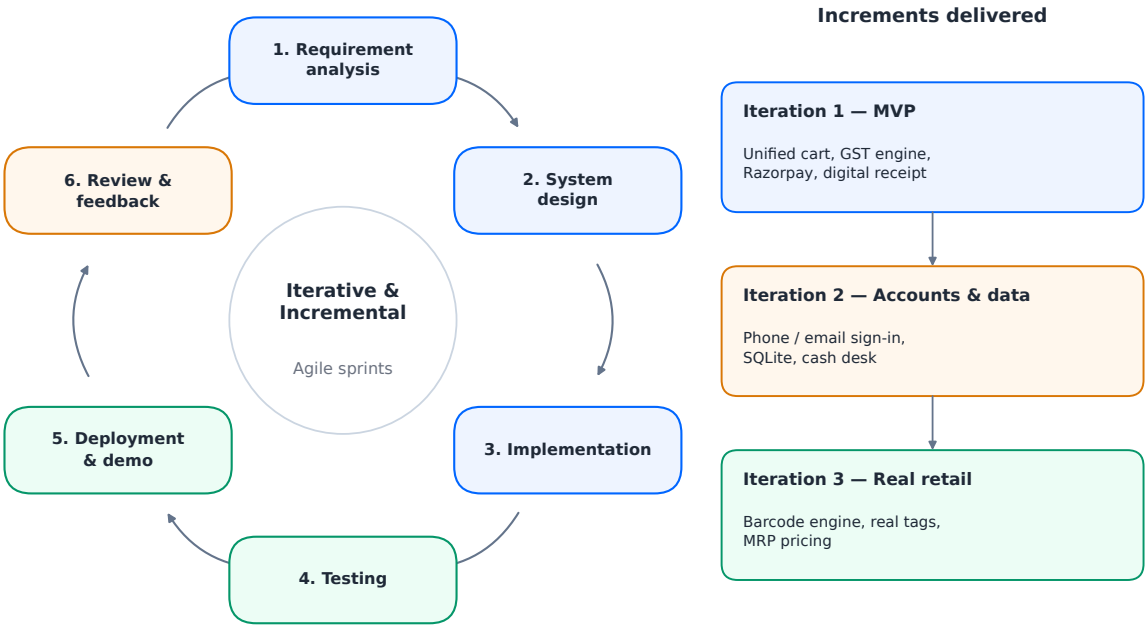


Figure 2.2: Iterative and incremental SDLC with the three delivered increments

2.4.2 UML Sequence Diagram

Figure 2.4 traces an online payment across the customer, the React frontend, the Express backend, the SQLite database and Razorpay. The amount is computed only on the server (steps 3–5), and no invoice is written unless the signature check in step 12 succeeds. The `alt` fragment shows both outcomes.

2.4.3 Activity Diagram

Figure 2.5 models the customer's checkout. A scan loops back on an invalid or out-of-stock code. The bag grows until the customer proceeds. The flow then forks on the payment method: online payments retry until the signature verifies, while cash bills wait for the cashier. Both paths merge at a single *Paid* receipt.

2.4.4 Class Diagram

Figure 2.6 is the domain model. A *Customer* owns shopping sessions; a session is composed of *CartLine*s that refer to *Product*s, and checks out as at most one *Transaction* (bill). Cash bills are collected within a *CashShift* opened by a *Cashier*. Transactions use the stateless *PricingEngine* and *PaymentGateway* services.

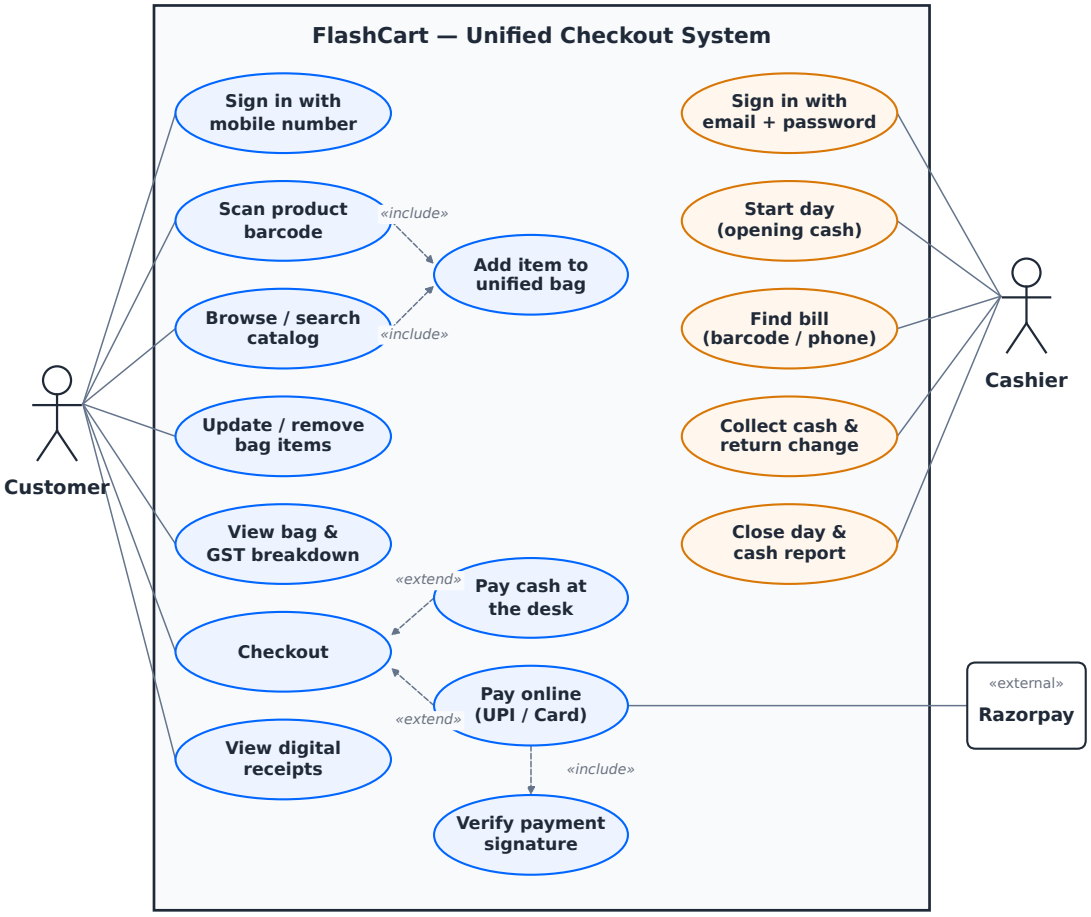


Figure 2.3: Use case diagram

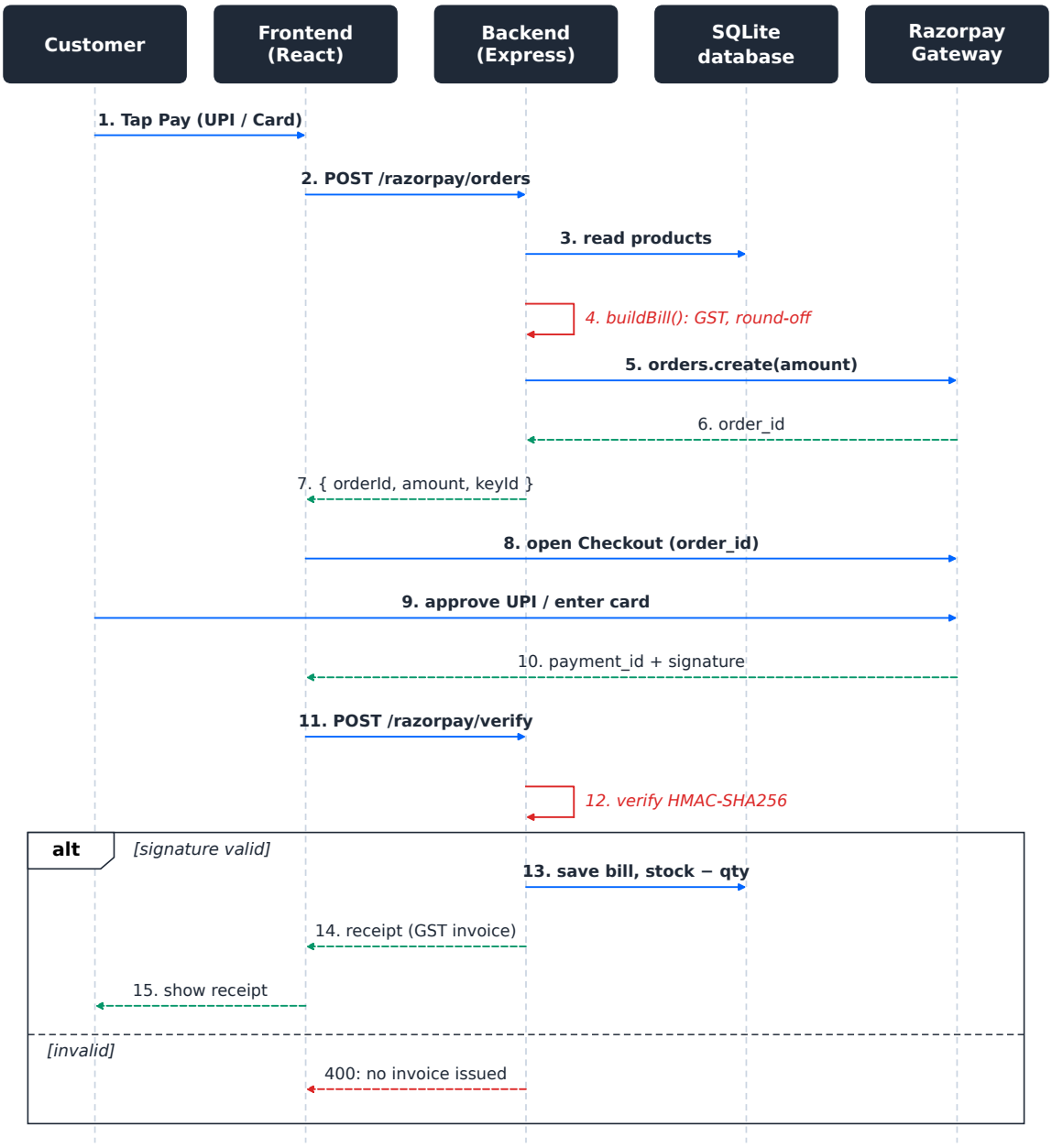


Figure 2.4: UML sequence diagram: checkout and Razorpay payment flow

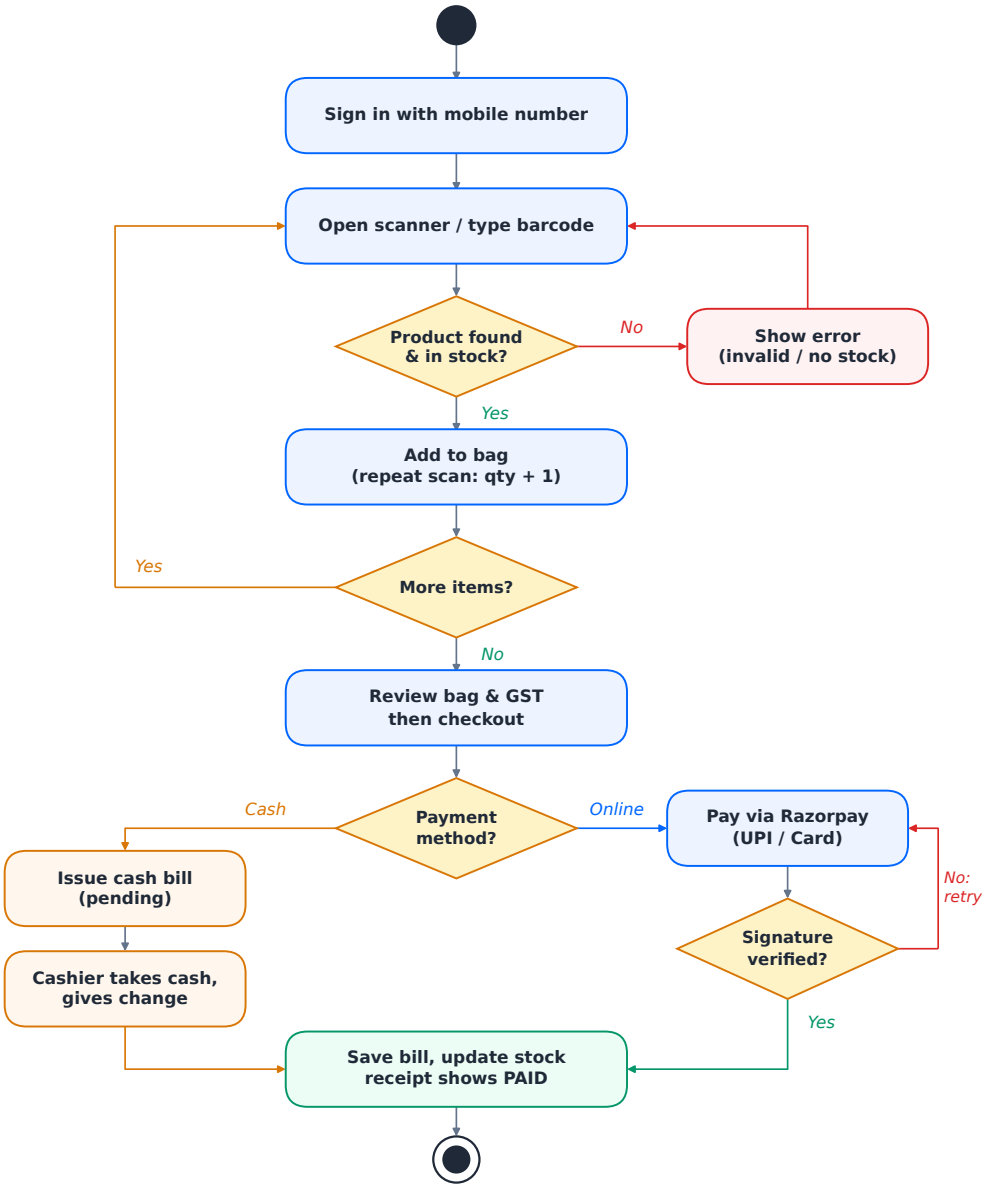


Figure 2.5: Activity diagram: customer checkout flow

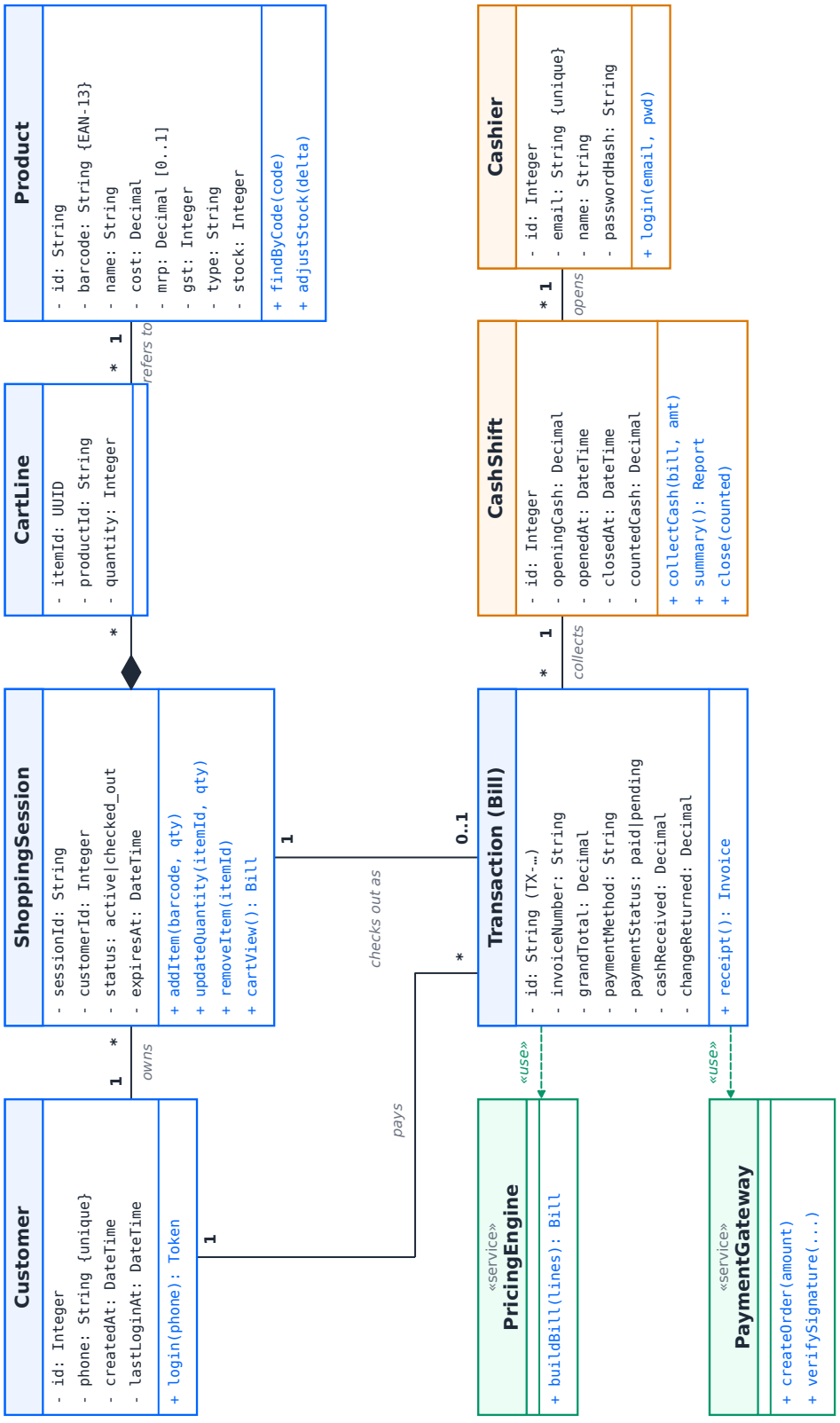


Figure 2.6: Class diagram: FlashCart domain model

Chapter 3

Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** the GST engine: slab totals, CGST/SGST split, round-off and MRP back-calculation.
- **Integration Testing:** 17 automated API tests (`node:test`) on the real Express app with an in-memory SQLite database: roles, cart, stock limits, Razorpay signatures (forged rejected, replays idempotent) and a full cash-desk day.
- **Performance Testing:** scripted load (90 customers $\times$ 3 scans, 90 cash bills, 50 collections) and barcode decode timing on photos of real tags.
- **User Testing:** browser end-to-end runs, a Razorpay test checkout, and real tags (Puma polo, Mufti shirt, JPG parfum).

3.2 Performance Metrics

Metric	**Target**	**Achieved**
Add-to-bag API latency (p95)	$<$ 100 ms	0.45 ms
Cash collection API latency (p95)	$<$ 100 ms	1.13 ms
Barcode decode per frame (WebAssembly)	$<$ 100 ms	25–35 ms
Real product tags decoded	3 / 3	3 / 3
Bill total vs. printed MRP	exact	exact
Automated tests passing	100%	17 / 17
Initial JavaScript download (gzip)	$<$ 150 KB	77.8 KB

Table 3.1: Performance results summary (Apple M4, server-side latencies)

3.3 Analysis

- **Objectives:** all five met. Scanning, which *failed* in the mid-term MVP, now reads real retail tags, and real-tag bills equal the printed MRP.
- **Challenges and fixes:** the MVP's QR-oriented scanner missed 1D barcodes, so we moved to native / ZXing-C++ decoding. MRP tags include GST, so we added tax-inclusive pricing. Data was lost on restart, so we added SQLite.

Chapter 4

Conclusion

4.1 Summary of Work

FlashCart delivers a working unified checkout for multi-showroom retail. Customers sign in with a phone number, scan real barcodes into one bag and pay once by UPI, card or cash. Cashiers get a focused cash desk with change calculation and a reconciled end-of-day total. Data is persisted in SQLite, and the rules are covered by automated tests.

4.2 Key Findings

- **Finding 1:** the EAN-13 codes already on tags are enough to run checkout.
- **Finding 2:** pricing must accept MRP-inclusive tags.
- **Finding 3:** server-side pricing and signature checks keep the client untrusted.

4.3 Future Work and Recommendations

1. **Inventory management:** a per-showroom inventory module for goods received, transfers and adjustments. It would add low-stock alerts and reorder suggestions, batch and expiry tracking for food, and an admin panel to add products (barcode, cost, MRP, GST, type), plus sales analytics and a daily stock reconciliation.
2. **Hardware smart cart:** a physical trolley with a mounted barcode scanner (or RFID reader), a Raspberry Pi / ESP32 and a touchscreen, joined to the same FlashCart session over Wi-Fi. A load cell (HX711) would check the measured weight against the scanned items to catch unscanned goods, and the customer could pay on the cart itself.
3. **Security:** SMS OTP for phone sign-in, Razorpay webhooks, returns and refunds.
4. **Scale:** PostgreSQL, cloud deployment and a pilot in one mall wing.

4.4 Final Remarks

The project covered the full software engineering cycle, from real retail requirements and UML models to tested increments. Letting demo feedback reshape the design made FlashCart simpler and more useful.